



SYSTEM DESIGN

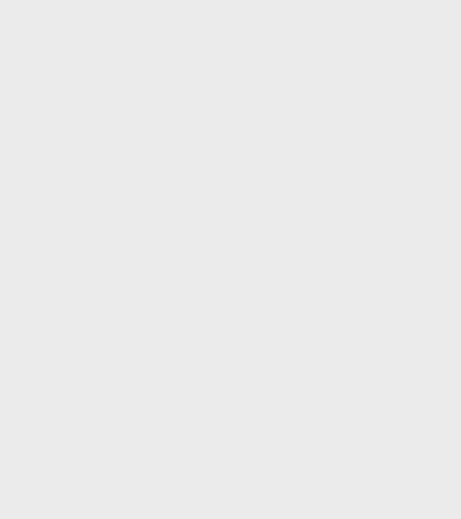
Twitter





FUNCTIONAL

- Create tweets(text and images)
- View a personalized home timeline
- Follow/Be followed



NON-FUNCTIONAL

- 
- Available
 - Scalable
 - Low latency
- 



ENTITIES

Nouns

01 User

02 Tweet

03 Follower



ENDPOINTS

POST /api/v1/tweets

```
{  
  "text": "string",  
  "parentID": "tweet_id (optional)",  
}
```

PUT /api/v1/users/:user_id/follow

GET /api/v1/timelines/home

Query Parameters: - limit:
int (default 20, max 50) -
cursor: string (pagination
token)



WAIT WAIT WAIT

We missed one endpoint.
Which one was it?

Lets revisit our functional
requirements





FUNCTIONAL

- Create tweets(text and **images**)
- View a personalized home timeline
- Follow/Be followed



NON-FUNCTIONAL

- Available
- Scalable
- Low latency

ENDPOINTS

```
POST /api/v1/tweets
```

```
{  
  "text": "string",  
  "media_ids": ["string"],  
  "parentID": "tweet_id (optional)",  
}
```

```
POST /api/v1/upload
```

```
PUT /api/v1/users/:user_id/follow
```

```
GET /api/v1/timelines/home
```

```
Query Parameters: - limit:  
int (default 20, max 50) -  
cursor: string (pagination  
token)
```

UPLOAD

The Naïvé way - Approach 1

Client

- POST /upload (multipart/form-data)
- ECS service receives file
- ECS uploads to S3 using AWS SDK

Problem with this?

Two uploads, one extra network call for each upload

The Naïvé way - Approach 2

Client directly uploads to S3

Problem with this?

Anyone can spam S3 with random images, not regulated.

RECOMMENDED WAY

Pre-signed URL

A presigned URL is a **temporary**, cryptographically signed S3 URL **generated** by the backend that allows a client to upload a specific object directly to S3 for a limited time.

Client

- GET /media/upload-url
- Backend validates user & generates presigned S3 URL
- Client uploads directly to S3 using that URL
- Backend stores metadata after successful upload

GET /media/upload-url

Response:

```
{
  "upload_url": "https://s3.amazonaws.com/...",
  "media_id": "med_abc123",
  "cdn_url": "https://cdn.example.com/med_abc123.jpg"
}
```

Why This Works

- No file traffic on backend
- No double upload
- Horizontally scalable
- Time-limited access (5-15 mins)
- Scoped to specific object key
- Prevents arbitrary S3 spam

ENDPOINTS

POST /api/v1/tweets

```
{  
  "text": "string",  
  "media_ids": ["string"],  
  "parentID": "tweet_id (optional)",  
}
```

GET /api/v1/media/upload-url

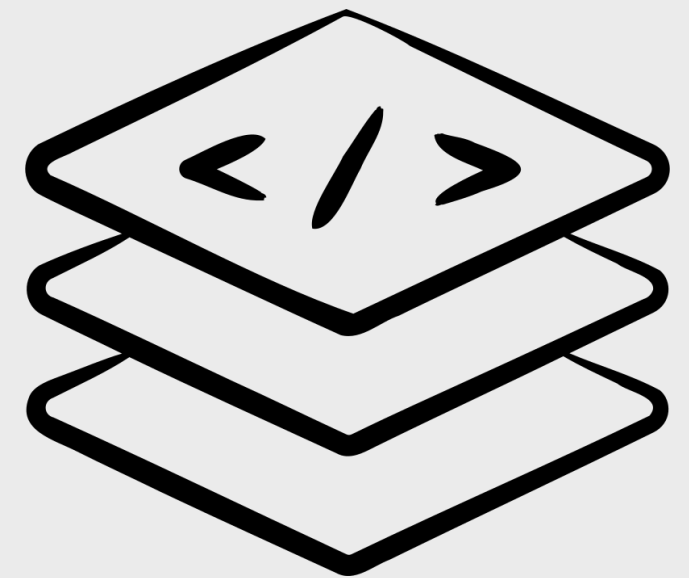
PUT /api/v1/users/:user_id/follow

GET /api/v1/timelines/home

Query Parameters: - limit:
int (default 20, max 50) -
cursor: string (pagination
token)

DATABASE

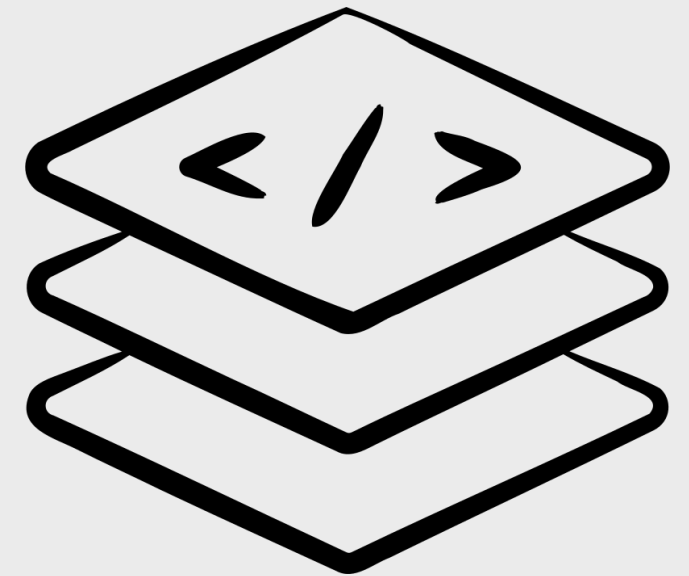
There is no “right” choice



DATABASE

Our data

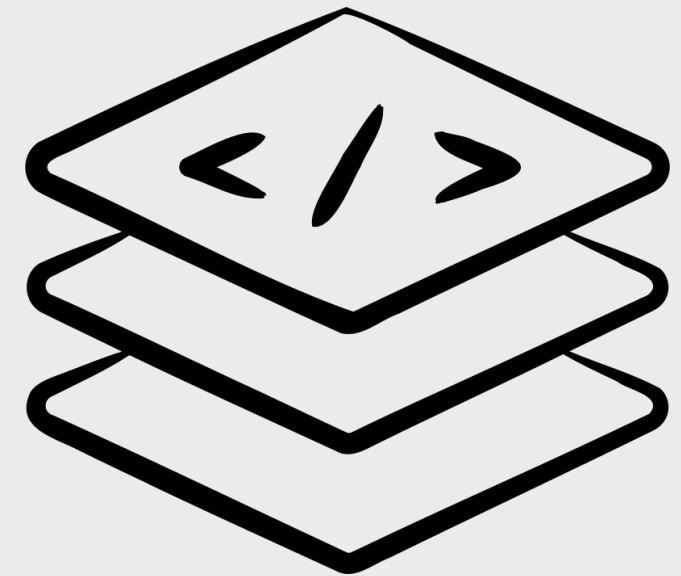
- Predictable access patterns: always query by known keys (user_id, post_id)
- No complex joins required
- Range queries on sort keys
- No complex joins, though we have posts and followers, our scale is too big to be joining the two.



DATABASE

System Requirements:

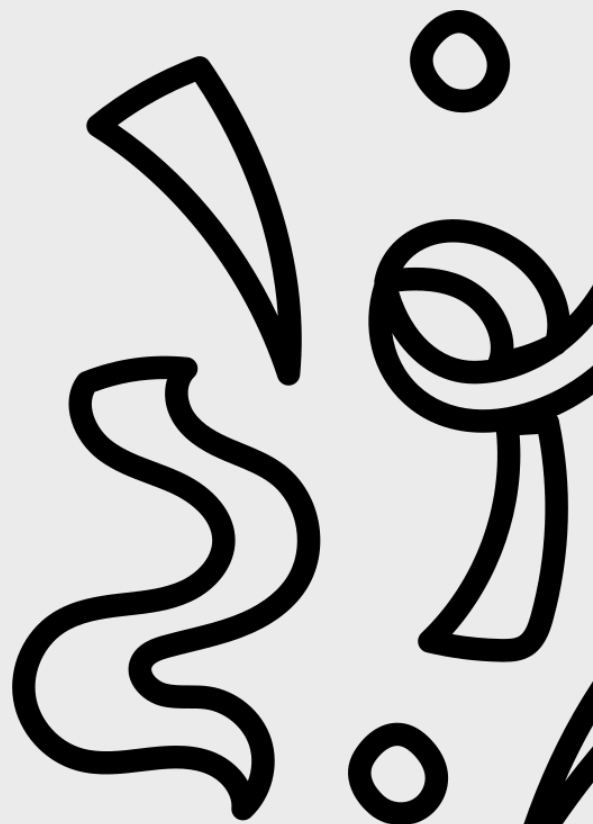
- Availability > Consistency (AP system)
- We need consistency for following/unfollowing
- Read heavy – more people read tweets than write them.
- But its write heavy too.
- Our scale is HUGE



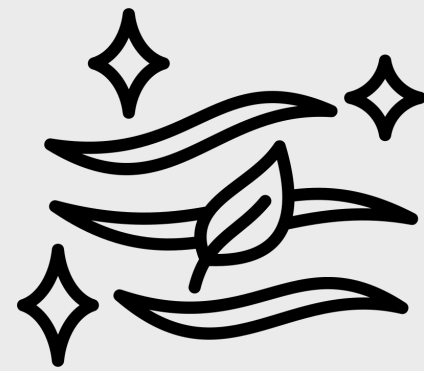


DATABASE I'M GOING WITH

Dynamo DB



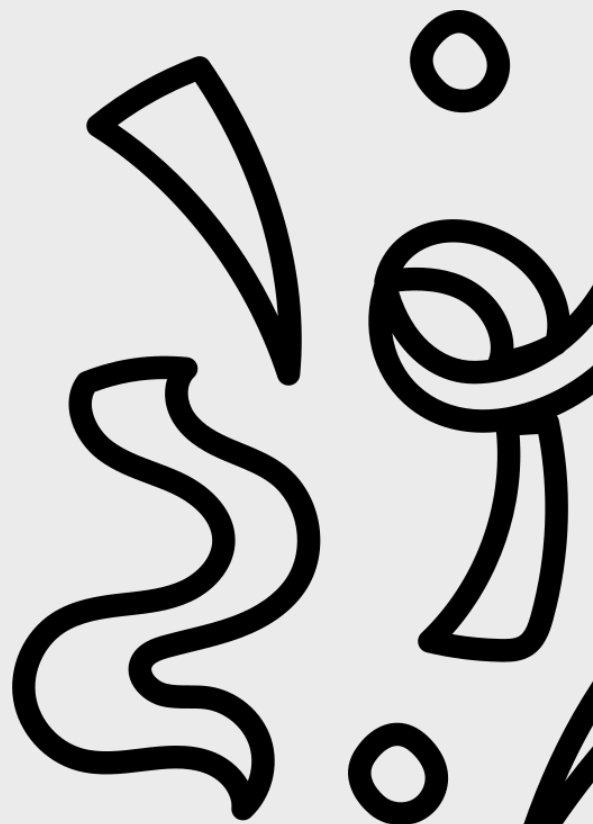
BUT





BUT

you can also use PostgreSQL





AGW VS ALB



AGW VS ALB

500M timeline reads/min = 8.3M requests/sec

API Gateway pricing: \$3.50 per million requests

Cost: 8.3M req/sec × 60 sec × 60 min × 24 hrs = 717 billion req/day

API Gateway cost: ~ **\$2.5M/day** = **\$75M/month** 💰

ALB cost: ~ **\$5K/month** (fixed hourly + LCU charges)

AGW VS ALB

But AGW gives throttling,
authentication, CORS handling
etc out of the box,
which ALB does not provide

Do you all know a service that
provides these missing services and
more? (Yes I sound like an AWS
Advertisement)

AWS WAF

(Web Application Firewall)

Attached to CloudFront/ALB

Blocks: Abusive IPs, bots, DDoS attempts

Rule: "Max 2000 requests per 5 minutes per IP"

Protects before traffic reaches our services





SERVICES

01

tweet flow

02

follow flow

03

view timeline flow





SERVICES

01

tweet service

02

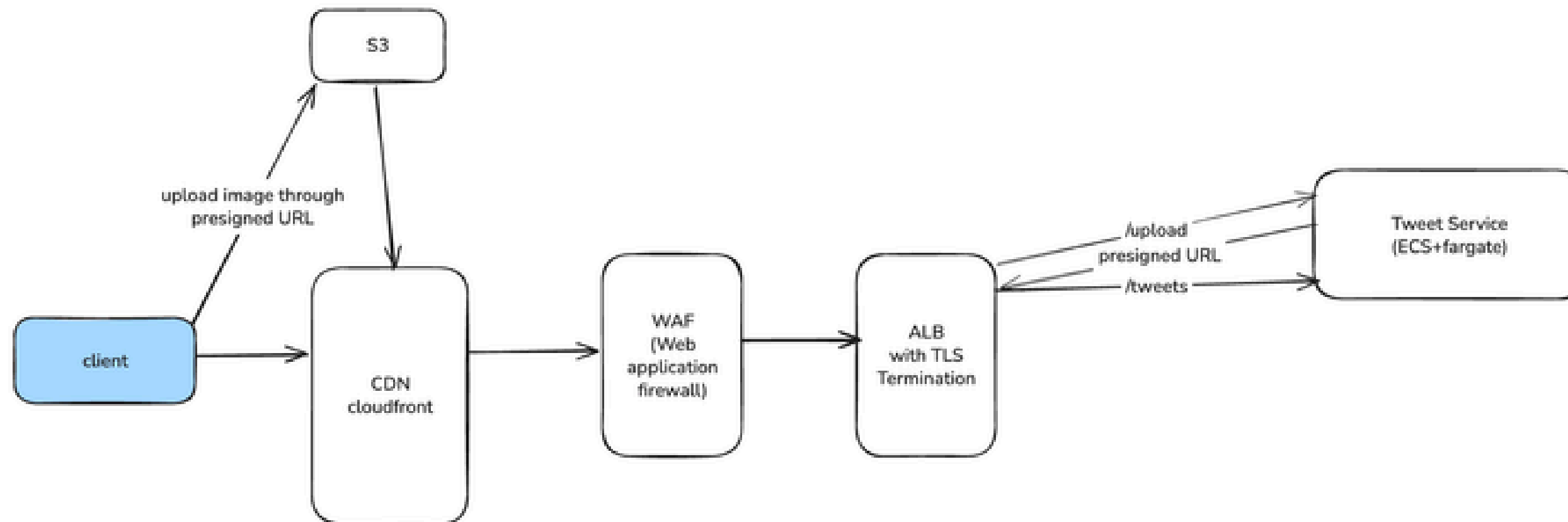
follow service

03

view timeline service



TWEET SERVICE





TWEET TABLE

Two access patterns

- user writes/edits by tweetID
 - query all tweets of one user
- 



TWEET TABLE

we query by tweetID more than by userID, so we need our partition key to be by tweetID

Partition Key (PK): TWEET#{tweetID} <--- namespacing pattern

Sort Key (SK): TWEET (this can be anything)

Create GSI on userID and creation time

GSI_PK: USER#84291

GSI_SK: 2026-02-17T10:15:30Z





TWEET TABLE

```
{  
  "PK": "POST#01HN9X8K3F7YQJ2W5Z6A8B9C1D",    <-----ULID  
  "SK": "POST",  
  
  "post_id": "01HN9X8K3F7YQJ2W5Z6A8B9C1D",  
  "author_id": "USER#84291",  
  "content": "Distributed systems are about managing tradeoffs.",  
  "created_at": "2026-02-17T10:15:30Z",  
  "media_refs": [],  
  "celebrity_flag": false  
}
```



FOLLOW SERVICE





FOLLOW TABLE

Two main access patterns:

- User A follows User B
 - Query all followers of a user (we will see this in timeline)
 - Query all users a user is following (if i want to see my following)
- 



FOLLOW TABLE

Each follow relationship is stored twice:

- For "who I follow"
- For "who follows me"

This gives efficient queries in both directions.





FOLLOW TABLE

Following Item (who I follow)

PK: USER#84291
SK: FOLLOWING#12345

User 84291 follows User 12345

Follower Item (who follows me)

PK: USER#12345
SK: FOLLOWER#84291

User 84291 is a follower of User 12345





FOLLOW TABLE

```
{  
  "PK": "USER#84291",  
  "SK": "FOLLOWING#12345",  
  
  "user_id": "84291",  
  "target_user_id": "12345",  
  "created_at": "2026-02-17T10:20:00Z"  
}
```





FOLLOW TABLE

```
{  
  "PK": "USER#84291",  
  "SK": "FOLLOWING#12345",  
  
  "user_id": "84291",  
  "target_user_id": "12345",  
  "created_at": "2026-02-17T10:20:00Z"  
}
```

What if a user has 10M followers?





FOLLOW TABLE

Instead of

PK: USER#12345

We do:

PK: USER#12345#SHARD#03

Now we distribute **writes across multiple shards.**





HOW TO SHARD?

Choose N shards (example: 10 or 100)

When someone follows user 12345:

```
shard = hash(follower_id) % 10
```

Then:

PK: USER#12345#SHARD#7

SK: FOLLOWER#84291





PROBLEM WITH APPROACH

Two writes, one write can fail , leading to inconsistent state





OTHER SCHEMA DESIGNS

Single Table + Global Secondary Index (GSI)

Instead of writing:

FOLLOWING item

FOLLOWER item

We write only one item, and GSI projects it differently.





OTHER SCHEMA DESIGNS

Single Table + Global Secondary Index (GSI)

```
{  
  "PK": "USER#84291",  
  "SK": "FOLLOW#12345",  
  
  "follower_id": "84291",  
  "following_id": "12345",  
  "entity_type": "FOLLOW",  
  
  "GSI1_PK": "USER#12345",  
  "GSI1_SK": "FOLLOWER#84291",  
  
  "created_at": "2026-02-17T10:30:00Z"  
}
```





OTHER SCHEMA DESIGNS

Single Table + Global Secondary Index (GSI)

Advantages

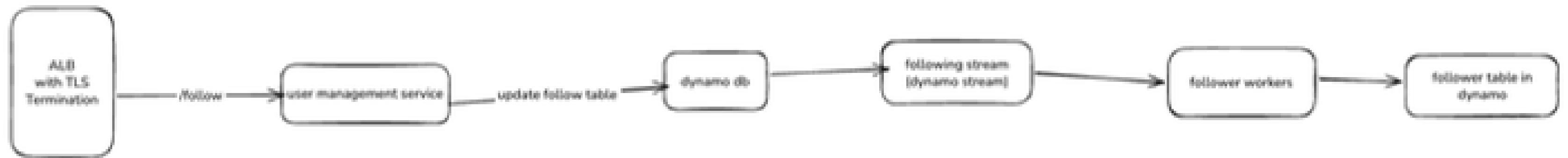
- Only one write per follow
- No duplication logic in app layer

Disadvantages

- More complex, in every write to handle hot partition problem, we need to compute the shard in the other fields
- 

OTHER SCHEMA DESIGNS

Two Separate Tables + Stream Replication





OTHER SCHEMA DESIGNS

Two Separate Tables + Stream Replication

Advantages

- separation of concerns
- independent scaling

Disadvantages

- eventual consistency
 - operational overhead
- 



TIMELINE SERVICE

Naive Approach — Read-Time Fanout

Get list of users I follow

For each user:

Query their latest posts

Merge all results

Sort by timestamp

Return top N





TIMELINE SERVICE

Naive Approach — Read-Time Fanout

Problem 1 – Too many reads

If a user follows 500 people:

→ 500 queries to DynamoDB

Even if each query returns only 10 posts:

→ 500 read operations per feed request



Problem 2 – Merge Complexity

You now must:

- Merge K sorted lists
- Maintain pagination state across K sources
- Track “last evaluated key” for each followee



TIMELINE SERVICE

Write time Fanout

What if instead of checking at runtime, we precompute the tweets a user needs to see?



ENTITIES

Nouns

01 User

02 Tweet

03 Follower

04 Timeline





TIMELINE SERVICE

Write time Fanout

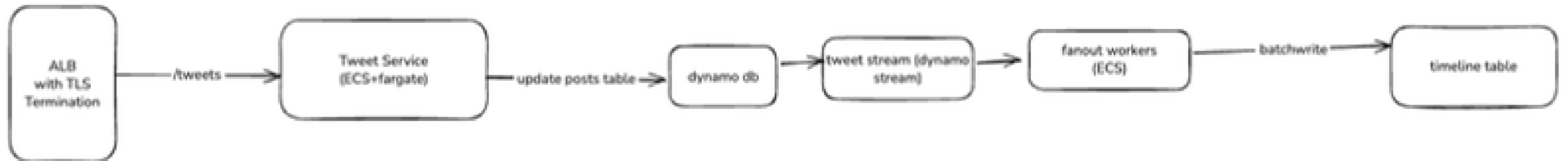
When a user creates a post:

Push that post into the timeline of all their followers.

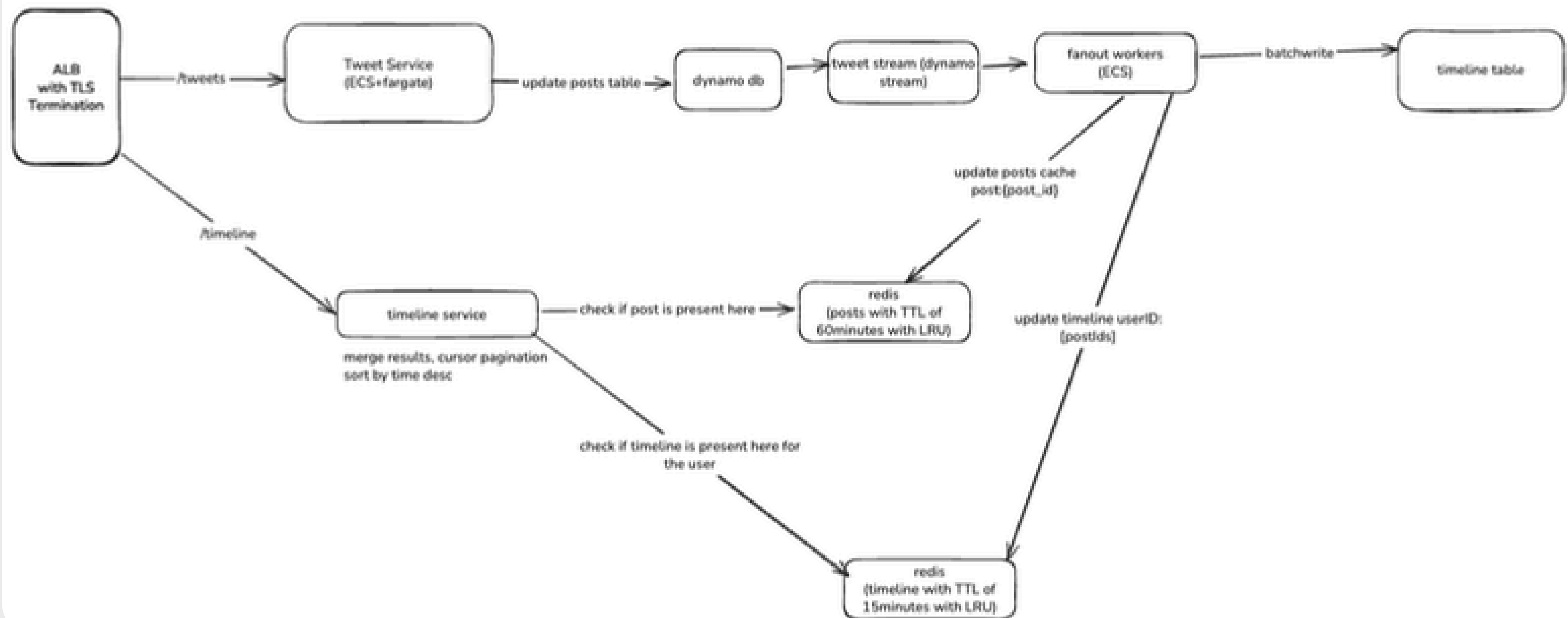
TIMELINE SERVICE

Write time Fanout

Lets revisit our /tweets endpoint



TIMELINE SERVICE





TIMELINE SERVICE

Write time Fanout

What about celebrities though?

If a celeb has 10M followers, do we have 10M writes in timeline?



TIMELINE SERVICE

Hybrid Fanout Approach

For mega-celebrities (>10M followers):

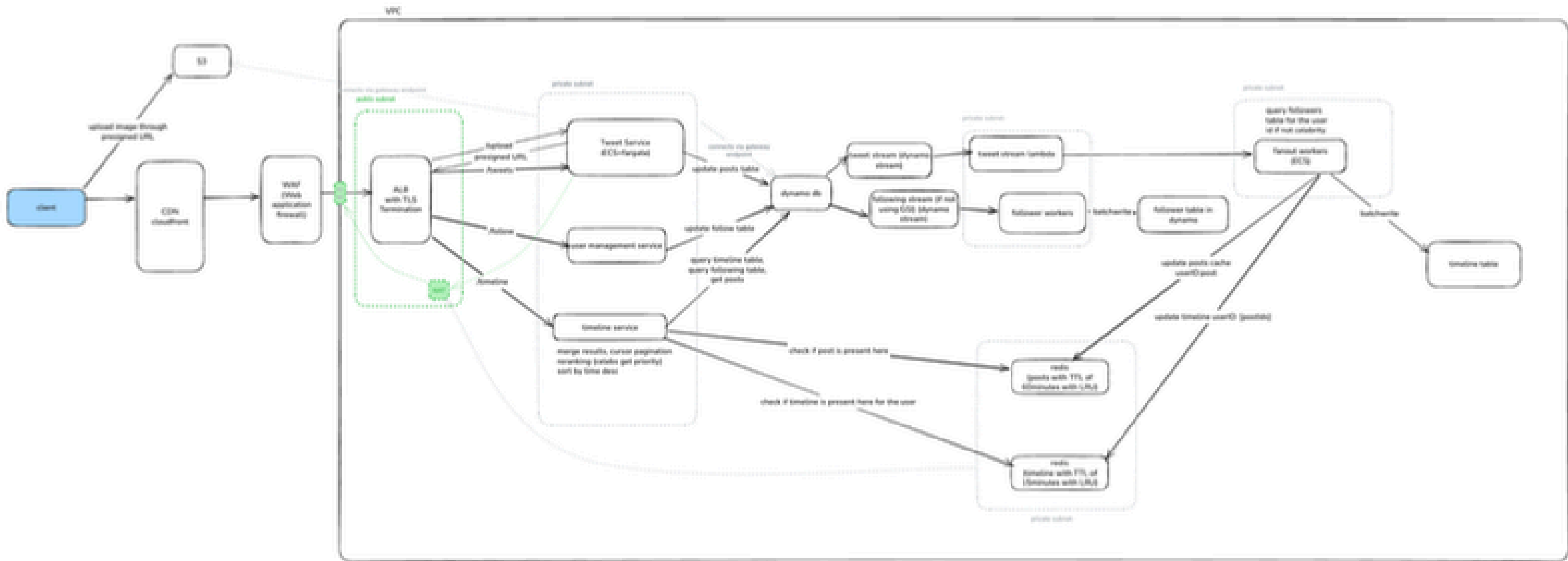
We do not fanout on write.

Instead:

Mark user as “celebrity mode”.

Store post only in Posts table.

Timeline Service merges posts at read time.





CONCLUSION

- everything we discussed is a small version of the massive scale twitter supports
 - in production, we would ideally replace ECS with EKS, use kinesis along with dynamo streams
- 



CONCLUSION

- there is no perfect solution to a system design problem, you just have to pick the cause that matters more





**THANK
YOU**